
RAPPORT DE TP : LIVE CODING 6

Architecture Distribuée, Résilience et Observabilité

Réalisé par :
METOGHE OBIANG Simplicie Dariel

Encadré par :
M. AMAMOU Ahmed

1 Résultats Expérimentaux et Captures d'Écran

Les tests ont été menés sur un serveur HTTP local (bibliothèque standard Python) et un client intégrant des mécanismes de fiabilité. Seuls les codes d'erreur HTTP réellement implémentés dans le serveur sont présentés ci-dessous.

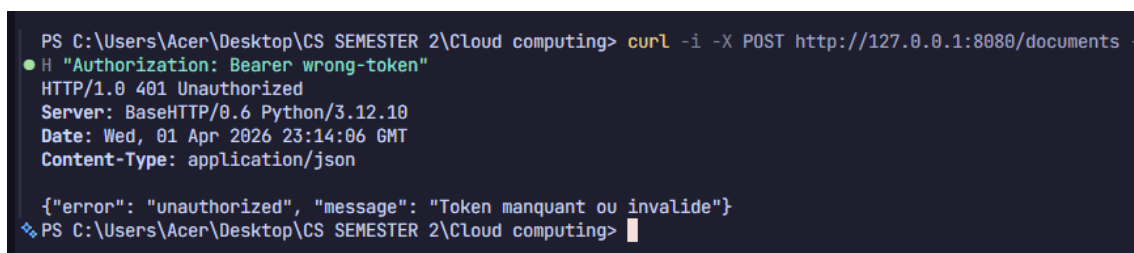
1.1 Codes HTTP implémentés et testés

Le serveur gère explicitement les codes suivants :

- **200 OK** – Réponse normale (GET /health)
- **201 Created** – Document créé avec succès (POST /documents)
- **400 Bad Request** – Corps vide, JSON invalide, champ manquant, titre trop long
- **401 Unauthorized** – Token absent ou invalide
- **404 Not Found** – Endpoint inexistant

Aucun code 5xx n'est généré volontairement dans cette version (absence de base de données externe).

1.2 Visualisation des Cas d'Erreur HTTP



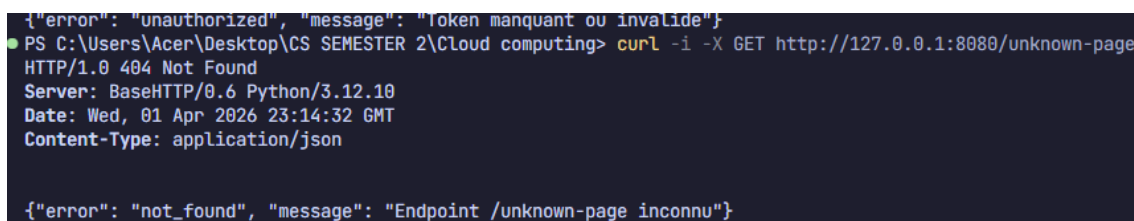
```
PS C:\Users\Acer\Desktop\CS SEMESTER 2\Cloud computing> curl -i -X POST http://127.0.0.1:8080/documents -
● H "Authorization: Bearer wrong-token"
HTTP/1.0 401 Unauthorized
Server: BaseHTTP/0.6 Python/3.12.10
Date: Wed, 01 Apr 2026 23:14:06 GMT
Content-Type: application/json

{"error": "unauthorized", "message": "Token manquant ou invalide"}
❖ PS C:\Users\Acer\Desktop\CS SEMESTER 2\Cloud computing>
```

Contexte : Tentative d'accès à une ressource protégée sans authentification.

Comportement : Le serveur rejette immédiatement la requête (HTTP 401) et ne procède à aucun traitement métier.

FIGURE 1 – Requête non authentifiée – bloquée sans traitement métier.



```
{\"error\": \"unauthorized\", \"message\": \"Token manquant ou invalide\"}
● PS C:\Users\Acer\Desktop\CS SEMESTER 2\Cloud computing> curl -i -X GET http://127.0.0.1:8080/unknown-page
HTTP/1.0 404 Not Found
Server: BaseHTTP/0.6 Python/3.12.10
Date: Wed, 01 Apr 2026 23:14:32 GMT
Content-Type: application/json

{"error": "not_found", "message": "Endpoint /unknown-page inconnu"}
```

Le serveur ne dispose d'aucun gestionnaire pour cette URL. Il retourne un code 404 avec un corps JSON explicite indiquant l'erreur.

FIGURE 2 – Gestion des routes non implémentées (HTTP 404).

```

PS C:\Users\Acer\Desktop\CS SEMESTER 2\Cloud computing> curl -i -X POST http://127.0.0.1:8080/documents -H "Authorization: Bearer secret-token-abc123" -H "Content-Type: application/json" -d '{"title": "My Long Title", "content": "Test"}'
HTTP/1.0 400 Bad Request
Server: BaseHTTP/0.6 Python/3.12.10
Date: Wed, 01 Apr 2026 21:06:28 GMT
Content-Type: application/json

{"error": "validation_error", "message": "'title' ne doit pas d'\u00e9passer 200 caract\u00e8res"}
PS C:\Users\Acer\Desktop\CS SEMESTER 2\Cloud computing>

```

Le serveur valide systématiquement les types, la présence des champs et les contraintes métier avant toute création.

FIGURE 3 – Validation stricte des entrées (HTTP 400) – aucune confiance dans les données client.

1.3 Démonstration de la Résilience Client

Le client implémente les politiques de fiabilité suivantes :

- Timeout configurable à **30 secondes** pour l'ensemble des requêtes
- Retry sur erreurs réseau et codes 5xx (502, 503, 504)
- Backoff exponentiel : $delay = \min(base \times 2^{attempt}, max_delay)$
- Jitter aléatoire pour éviter l'effet de foule
- Pas de retry sur les codes 4xx (hors 429)
- X-Request-ID généré par requête pour la traçabilité

Justification du timeout à 30 secondes : Ce délai permet de tolérer des latences réseau élevées ou des traitements serveur longs sans pour autant bloquer indéfiniment la ressource client. Une valeur plus faible (5-10s) aurait généré trop de faux positifs dans un environnement cloud réel où la latence peut varier considérablement.

```

PS C:\Users\Acer\Desktop\CS SEMESTER 2\Cloud computing> python .\client-v2.py
--- Lancement du client résilient ---
[acaa1f1b] GET http://127.0.0.1:8080/health
  Erreur réseau: [WinError 10061] No connection could be made because the target machine actively refused it
  Essai 1/4 échoué. Retry dans 1.0s...
[8cbfc47d] GET http://127.0.0.1:8080/health
  Erreur réseau: [WinError 10061] No connection could be made because the target machine actively refused it
  Essai 2/4 échoué. Retry dans 1.9s...
[14f6b532] GET http://127.0.0.1:8080/health
  Erreur réseau: [WinError 10061] No connection could be made because the target machine actively refused it
  Essai 3/4 échoué. Retry dans 1.9s...
[93e359d4] GET http://127.0.0.1:8080/health
  Erreur réseau: [WinError 10061] No connection could be made because the target machine actively refused it
  Essai 4/4 échoué. Retry dans 3.5s...
[8ba9c7af] GET http://127.0.0.1:8080/health
[57e590d5] POST http://127.0.0.1:8080/documents
  Erreur réseau: [WinError 10061] No connection could be made because the target machine actively refused it
  Essai 2/4 échoué. Retry dans 0.3s...
[85cc7121] POST http://127.0.0.1:8080/documents
  Erreur réseau: [WinError 10061] No connection could be made because the target machine actively refused it
  Essai 3/4 échoué. Retry dans 0.2s...
[8fb8a389] POST http://127.0.0.1:8080/documents
  Erreur réseau: [WinError 10061] No connection could be made because the target machine actively refused it
  Essai 4/4 échoué. Retry dans 4.8s...
[fbcd4ec5e] POST http://127.0.0.1:8080/documents
  Erreur réseau: [WinError 10061] No connection could be made because the target machine actively refused it
  ⚠ Échec définitif après 5 tentatives

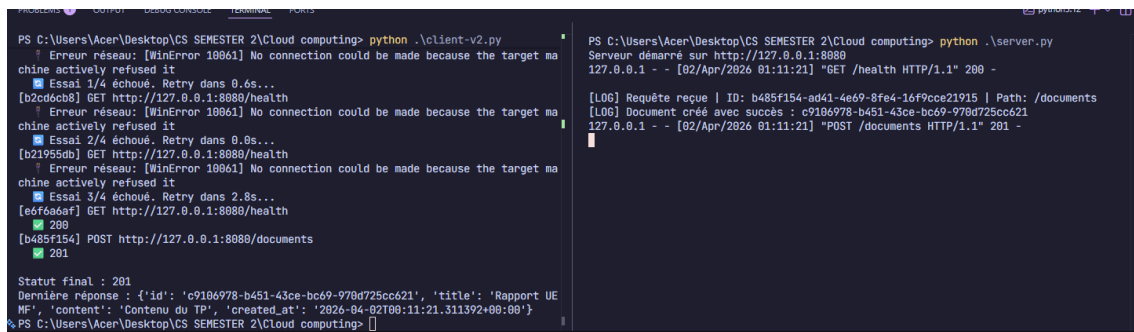
Statut final : None
Dernière réponse : {'error': '[WinError 10061] No connection could be made because the target machine actively refused it'}
PS C:\Users\Acer\Desktop\CS SEMESTER 2\Cloud computing>

```

Simulation d'une panne serveur temporaire. Le client exécute des tentatives de reconnexion avec un délai exponentiel et du *jitter*, évitant ainsi une surcharge du service lors de son redémarrage.

FIGURE 4 – Algorithme de Backoff Exponentiel avec Jitter lors d'une panne.

1.4 Traçabilité Distribuée (X-Request-ID)



```
PS C:\Users\Acer\Desktop\CS SEMESTER 2\Cloud computing> python .\client-v2.py
Erreur réseau: [WinError 10061] No connection could be made because the target machine actively refused it
Essai 1/4 échoué. Retry dans 0.6s...
[b2cd6cb8] GET http://127.0.0.1:8080/health
Erreur réseau: [WinError 10061] No connection could be made because the target machine actively refused it
Essai 2/4 échoué. Retry dans 0.8s...
[b21955db] GET http://127.0.0.1:8080/health
Erreur réseau: [WinError 10061] No connection could be made because the target machine actively refused it
Essai 3/4 échoué. Retry dans 2.8s...
[e6f6a6af] GET http://127.0.0.1:8080/health
200
[b485f154] POST http://127.0.0.1:8080/documents
201
Statut final : 201
Dernière réponse : {'id': 'c9104978-b451-43ce-bc69-970d725cc621', 'title': 'Rapport UE MF', 'content': 'Contenu du TP', 'created_at': '2026-04-02T08:11:21.311392+00:00'}
PS C:\Users\Acer\Desktop\CS SEMESTER 2\Cloud computing>

PS C:\Users\Acer\Desktop\CS SEMESTER 2\Cloud computing> python .\server.py
Serveur démarré sur http://127.0.0.1:8080
127.0.0.1 - - [02/Apr/2026 01:11:21] "GET /health HTTP/1.1" 200 -
[LOG] Requête reçue | ID: b485f154-ad41-4e69-8fe4-16f9cce21915 | Path: /documents
[LOG] Document créé avec succès : c9104978-b451-43ce-bc69-970d725cc621
127.0.0.1 - - [02/Apr/2026 01:11:21] "POST /documents HTTP/1.1" 201 -
```

Le client génère un UUID unique par requête et le transmet dans l'en-tête X-Request-ID. En environnement distribué réel, ce même identifiant est propagé à travers l'API Gateway, les services internes et les bases de données, permettant un débogage de bout en bout.

FIGURE 5 – Tracing distribué via Request-ID – condition minimale d'observabilité.

1.5 Bilan des codes HTTP gérés

Code HTTP	Déclencheur dans le code	Sécurité / Fiabilité
200	GET /health	Sonde de santé pour orchestration
201	POST valide	Création réussie d'une ressource
400	JSON invalide / champ manquant / titre trop long	Anti-injection, validation systématique
401	Token absent ou invalide	Authentification obligatoire avant traitement
404	Route inexistante	Pas d'indice sur l'existence des ressources

TABLE 1 – Récapitulatif des codes HTTP implémentés et testés.